

Cross-Framework Performance Comparison Report

Model: nvidia/Kimi-K2.5-NVFP4 (DeepSeekV3 arch, ~1T params)
GPU: NVIDIA B200 x8 (TP=8)
Execution: Decode mode, BS=1, ISL=4, OSL=1024
Block Type: MoE Decoder Layer (Layers 1-60)
Target Framework: vLLM v0.19.2rc1 (commit fe9c3d6c5f66c873d196800384ed6880687b9e52)
Baseline Framework: SGLang (commit 612785ffdcdf35552f1ed433a981d596ca9fe900)
Generated: 2026-05-14

Executive Summary

This report compares the per-block decode performance of **vLLM** (target) against **SGLang** (baseline) for the nvidia/Kimi-K2.5-NVFP4 model on 8x NVIDIA B200 GPUs with TP=8. The analysis examines a median MoE decoder block (one of 60 layers) at batch size 1 with input sequence length 4 and output sequence length 1024 in decode mode.

vLLM is **7,775 ns (8.6%) slower** per MoE decoder block than SGLang. The gap is driven by five root-cause categories: three negative (vLLM is slower) and one positive (existing vLLM advantage). Despite 4.0% more total compute in SGLang, SGLang achieves 48% more kernel overlap (39.1 us vs 26.3 us), resulting in lower wall time.

Metric	vLLM (Target)	SGLang (Baseline)	Delta
Wall time	97,919 ns	90,144 ns	+7,775 ns (+8.6%)
Total compute	124,253 ns	129,215 ns	-4,962 ns (-4.0%)
Overlap achieved	26,334 ns (21.2%)	39,071 ns (30.2%)	+12,737 ns (+48.4%)

Table 1: Wall time and overlap comparison per MoE decoder block.

Identified Issues & Improvement Plan

#	Issue	Impact (ns)	Difficulty	Priority
IMP-1	Unsharded shared expert + stream swap	-7,232	Med-High	HIGHEST
IMP-2a	Pre-convert routing bias to bfloat16	-1,984	Low	HIGH
IMP-2b	Eliminate MoE workspace zero-fill	-2,368	Medium	HIGH

#	Issue	Impact (ns)	Difficulty	Priority
IMP-3	Optimize Q concat+scale kernel	-1,696	Medium	MEDIUM
IMP-4	Attention micro-optimizations	-350	Low-Med	LOW
---	TOTAL RECOVERABLE	-13,630	---	---
POS-1	AllReduce+RMSNorm fusion (advantage)	+6,496	N/A	PROTECT
---	NET (if all applied)	-6,134 faster	---	---

Table 2: Ranked improvement plan. Negative deltas = vLLM improvement.

After all improvements combined with the existing AllReduce+RMSNorm fusion advantage, the estimated vLLM block time would be **~84,289 ns**, making vLLM approximately **6.5% faster** than SGLang.

Detailed Improvement Plans

HIGH IMPACT

IMP-1: Unsharded Shared Expert + Stream Swap

Problem

vLLM places the routed MoE pipeline (37.1 us, including 10.7 us of routing overhead for gate GEMM + top-8 selection) on the main CUDA stream, making it the critical path. The TP-sharded shared expert pipeline (22.4 us) runs on an auxiliary stream but finishes 14.7 us early, wasting overlap potential. SGLang inverts this arrangement: it runs an unsharded shared expert (29.9 us) on main stream and hides the routed MoE (32.9 us) on an alternate stream, achieving near-perfect dual-stream balance.

Root Cause

vLLM applies default tensor-parallel sharding to the shared expert, reducing its GEMM sizes by 8x per GPU. This makes the shared expert pipeline too short (22.4 us) to serve as a credible main-stream workload. SGLang explicitly sets `tp_size=1` for the shared expert when the FlashInfer backend is active, keeping the full-size GEMMs ([7168]->[4096] gate_up, [2048]->[7168] down) and producing a 29.9 us pipeline that closely matches the routed MoE duration.

Performance Impact

Metric	vLLM (Target)	SGLang	Delta
MoE main-stream critical path	41,471 ns	~34,432 ns	-7,039 ns
Routed MoE pipeline (main vs alt)	37,119 ns (main)	32,928 ns (alt)	N/A
Shared expert pipeline (alt vs main)	22,400 ns (alt)	29,887 ns (main)	N/A
Routing overhead on critical path	10,688 ns	0 ns (hidden)	+10,688 ns
Net MoE critical path delta			+7,232 ns

Implementation Guide

Step 1: Add `disable_tp` parameter to `DeepseekV2MLP`.

File: `vllm/model_executor/models/deepseek_v2.py`

```

class DeepseekV2MLP(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        intermediate_size: int,
        hidden_act: str,
        quant_config: QuantizationConfig | None = None,
        reduce_results: bool = True,
        is_sequence_parallel=False,
        disable_tp: bool = False,          # NEW PARAMETER
        prefix: str = "",
    ) -> None:
        super().__init__()
        _disable_tp = is_sequence_parallel or disable_tp
        self.gate_up_proj = MergedColumnParallelLinear(
            hidden_size, [intermediate_size] * 2,
            bias=False, quant_config=quant_config,
            disable_tp=_disable_tp,          # USE COMBINED
            prefix=f"{prefix}.gate_up_proj",
        )

```

Listing 1: Add disable_tp parameter to DeepseekV2MLP constructor.

Step 2: Set `disable_tp=True` for shared expert when using TRTLLM NVFP4 MoE backend.

File: `vllm/model_executor/models/deepseek_v2.py` (shared expert construction)

```

# Disable TP for shared expert when using FlashInfer TRTLLM MoE
_disable_shared_expert_tp = self._should_disable_shared_expert_tp()

self.shared_experts = DeepseekV2MLP(
    hidden_size=config.hidden_size,
    intermediate_size=intermediate_size,
    hidden_act=config.hidden_act,
    quant_config=quant_config,
    disable_tp=_disable_shared_expert_tp,  # NEW
    reduce_results=False,
    prefix=f"{prefix}.shared_experts",
)

```

Listing 2: Pass disable_tp flag when constructing shared expert.

Step 3: Swap stream assignments so shared expert runs on main stream and routed MoE on auxiliary stream.

File: `vllm/model_executor/layers/fused_moe/runner/moe_runner.py`

```
def _forward_impl_swapped_streams(self, layer, hidden_states,
                                router_logits, shared_experts_input):
    aux = self._shared_experts._stream
    aux.wait_stream(current_stream())

    # MAIN STREAM: Run shared expert
    shared_output = self._shared_experts._layer(shared_experts_input)

    # AUX STREAM: Run gate + routing + fused MoE experts
    with torch.cuda.stream(aux):
        if self.gate is not None:
            router_logits, _ = self.gate(hidden_states)
            fused_out = self.quant_method.apply(
                layer=layer, x=hidden_states, ...)

    # SYNC: wait for routed MoE to finish
    current_stream().wait_stream(aux)
    return shared_output, fused_out
```

Listing 3: Swapped dual-stream execution (mirrors SGLang pattern).

HIGH IMPACT

IMP-2a: Pre-convert Routing Bias to bfloat16

Problem

vLLM stores the MoE `e_score_correction_bias` tensor as float32 and converts it to bfloat16 on every forward pass, launching a 1,984 ns `bfloat16_copy_kernel` on the main stream. This is a pure overhead kernel that SGLang avoids by pre-converting the bias at model initialization time.

Root Cause

The FlashInfer TRTLLM MoE API requires the routing bias in bfloat16 format (see FlashInfer issue #2909). vLLM performs this conversion in the hot path at `trtllm_nvfp4_moe.py:334`, while SGLang sets the dtype at initialization in `deepseek_v2.py:301-305`.

Performance Impact

Metric	vLLM (Target)	SGLang	Delta
Routing bias copy kernel	1,984 ns	0 ns	+1,984 ns

Implementation Guide

Step 1: Pre-convert bias dtype at model initialization.

File: `vllm/model_executor/models/deepseek_v2.py`

```

if getattr(config, "topk_method", None) == "noaux_tc":
    _bias_dtype = torch.float32
    if quant_config is not None:
        from vllm.model_executor.layers.quantization.modelopt_fp4 import (
            ModelOptFp4Config,
        )
        if isinstance(quant_config, ModelOptFp4Config):
            _bias_dtype = torch.bfloat16 # Pre-convert
    self.gate.e_score_correction_bias = nn.Parameter(
        torch.empty(config.n_routed_experts, dtype=_bias_dtype)
    )

```

Listing 4: Pre-convert routing bias dtype at initialization.

Step 2: Remove per-forward conversion in `trtllm_nvfp4_moe.py`.

```

# REMOVE the per-forward conversion:
# if e_score_correction_bias is not None:
#     e_score_correction_bias = e_score_correction_bias.to(torch.bfloat16)

# Replace with assertion:
if e_score_correction_bias is not None:
    assert e_score_correction_bias.dtype == torch.bfloat16

```

Listing 5: Remove per-forward dtype conversion.

HIGH IMPACT

IMP-2b: Eliminate MoE Workspace Zero-Fill

Problem

vLLM's MoE pipeline launches a `vectorized_elementwise_kernel` (FillFunctor, 2,368 ns) to zero the MoE output workspace buffer before expert computation. This happens internally within FlashInfer's MoE API. SGLang avoids this by allocating the output buffer with `torch.empty()` (no zeroing) and relying on the finalize kernel to fully overwrite all output elements.

Root Cause

vLLM uses the FlashInfer modular kernel API path that triggers internal workspace zeroing. The zero-fill is launched by FlashInfer's C++/CUDA code before expert computation begins. SGLang's direct wrapper avoids this by externally managing the output buffer.

Performance Impact

Metric	vLLM (Target)	SGLang	Delta
Workspace zero-fill kernel	2,368 ns	0 ns	+2,368 ns

Implementation Guide

Step 1: Pre-allocate output buffer externally without zeroing.

File: `vllm/model_executor/layers/fused_moe/experts/trtllm_nvfp4_moe.py`

```
# Pre-allocate output buffer without zeroing (SGLang pattern)
output = torch.empty(
    hidden_states.shape[0], hidden_states.shape[1],
    dtype=hidden_states.dtype, device=hidden_states.device,
)
# Pass pre-allocated output to FlashInfer kernel
flashinfer.fused_moe.trtllm_fp4_block_scale_routed_moe(
    ...,
    output=output,
    skip_workspace_zeroing=True, # If available
)
```

Listing 6: External buffer allocation to avoid internal zero-fill.

MEDIUM IMPACT

IMP-3: Optimize Q Concat+Scale Kernel

Problem

vLLM launches an explicit torch.compile-generated Triton kernel to concatenate the absorbed query components (q_nope_out[512] + q_pe[64] -> 576 dims) and apply softmax scaling before the MLA decode attention kernel. This adds 1,696 ns on the critical path. SGLang fuses this operation inside the TRT-LLM MLA attention kernel, which natively accepts separate Q components.

Root Cause

vLLM's MLA code explicitly concatenates Q before calling the attention backend (mla_attention.py:740-748). The _DecodeConcatQuantFP8 class uses torch.cat which torch.compile decomposes into a Triton pointwise kernel. SGLang passes separate tensors directly to the backend's concat_mla_absorb_q CUDA kernel (warp-vectorized, optimized for the 512+64 shape).

Performance Impact

Metric	vLLM (Target)	SGLang	Delta
Q concat+scale Triton kernel	1,696 ns	0 ns (fused)	+1,696 ns

Implementation Guide

Step 1: Register optimized concat as custom op to prevent torch.compile decomposition.

File: vllm/model_executor/layers/attention/mla_attention.py

```

@torch.library.custom_op("vllm::concat_mla_q", mutates_args=())
def concat_mla_q(
    q_nope: torch.Tensor,    # [..., 512]
    q_pe: torch.Tensor,      # [..., 64]
) -> torch.Tensor:
    return torch.cat([q_nope, q_pe], dim=-1)

@concat_mla_q.register_fake
def _(q_nope, q_pe):
    out_dim = q_nope.shape[-1] + q_pe.shape[-1]
    return torch.empty(
        *q_nope.shape[:-1], out_dim,
        dtype=q_nope.dtype, device=q_nope.device,
    )

```

Listing 7: Custom op to prevent torch.compile Triton generation.

Step 2: Use the custom op in the FlashInfer MLA backend.

File: vllm/v1/attention/backends/mla/flashinfer_mla.py

```

def forward_mqa(self, q, kv_c_and_k_pe_cache, attn_metadata, layer):
    ...
    if isinstance(q, tuple):
        q_nope, q_pe = q
        q = torch.ops.vllm.concat_mla_q(q_nope, q_pe) # Custom op

```

Listing 8: Replace torch.cat with custom op in MLA backend.

LOW IMPACT

IMP-4: Attention Micro-Optimizations

Problem

Several attention-phase kernels show small duration differences between frameworks, collectively adding +350 ns. The two actionable items are: (a) Q-A RMSNorm split into 2 Triton kernels by torch.compile (+223 ns), and (b) KV cache update performing concat+FP8 quant+write in one kernel vs SGLang's simpler write-only kernel (+415 ns), partially offset by vLLM's superior W_UK BMM layout optimization (-736 ns).

Performance Impact

Component	vLLM (ns)	SGLang (ns)	Delta (ns)
Q-A RMSNorm (2 Triton vs 1 FlashInfer)	4,191	3,968	+223
KV cache update	2,079	1,664	+415
W_UK BMM layout (vLLM advantage)	2,880	3,616	-736
Same-kernel variance (noise)	+776	-1,024	+448
Net			+350

Implementation Guide

- Step 4a:** Use standalone FlashInfer RMSNorm for Q-A norm instead of allowing torch.compile to split it into 2 Triton kernels. Register the layernorm as a custom op or graph-break point.
- Step 4b:** Fuse RoPE + FP8 quantization into a single kernel (matching SGLang's RopeQuantizeKernel pattern) to simplify the KV cache write kernel. This is lower priority since it requires modifying the attention pipeline.

Existing Advantage: AllReduce + RMSNorm Fusion (POS-1)

vLLM's single largest advantage is the AllReduce+RMSNorm fusion, saving **6,496 ns** per block. vLLM fuses AllReduce + RMSNorm + residual-add into a single Lamport-synchronized kernel at two points per block. SGLang's allreduce fusion is disabled on this B200 configuration (cuMemCreate probe failed), requiring separate AllReduce + fused_add_rmsnorm kernels. This advantage should be protected and maintained across all configurations.

Metric	vLLM (Target)	SGLang	Delta
Input norm (AR+RMSNorm+res)	7,040 ns (fused)	9,984 ns (2 kernels)	-2,944 ns
Post-attn norm (AR+RMSNorm+res)	7,904 ns (fused)	11,456 ns (2 kernels)	-3,552 ns
Total fusion saving			-6,496 ns

Appendix A: Operation-by-Operation Matching

Complete matched operations between vLLM (Trace 0, Target) and SGLang (Trace 1, Baseline) for the median MoE decoder block. Delta = SGLang duration - vLLM duration (negative = SGLang faster).

Attention Phase (Groups 1-12)

Gr p	Logical Operation	vLLM Kernel	vS	vLLM D(ns)	SGL Kernel	sS	SGL D(ns)	Delta
1	Input RMSNorm + Residual (+ AllReduce in vLLM)	allreduce_fusion_kernel_oneshot_lamport	23	7,040	fused_add_rmsnorm	7	3,328	-3,712
2	Fused QKV-A Projection	fused_a_gemm_kernel	23	7,584	fused_a_gemm_kernel	7	7,520	-64
3	Q-A LayerNorm (RMSNorm)	triton_poi + triton_red (COMBINED)	23	4,191	flashinfernormkernels rmsnorm	7	3,968	-223
4	KV-A LayerNorm	(fused into Group 6)	--	0	flashinfernormkernels rmsnorm	3488	1,568	N/A
5	Q-B Projection GEMM	nvjet_sm100_tst_16x64	23	5,792	nvjet_sm100_tst_16x64	7	5,504	-288
6	KV-A Norm + RoPE	triton_poi_fused_add_clone	23	2,112	RopeQuantizeKernel	7	2,176	+64
7	KV Cache Update	concat_and_cache_mla	23	2,079	set_mla_kv_buffer	7	1,664	-415
8	W_UK/W_kc Absorbed BMM	nvjet_sm100 (NNT)	23	2,880	nvjet_sm100 (TNT)	7	3,616	+736
9	Q concat + softmax scale (vLLM ONLY)	triton_poi_fused_to_copy_cat_clamp_mul	23	1,696	---	--	---	N/A
10	MLA Decode Attention	fmhaSm100fKernel	23	9,536	fmhaSm100fKernel	7	9,376	-160
11	W_UV/W_vc Absorbed BMM	nvjet_sm100_tst_8x64	23	4,000	nvjet_sm100_tst_8x64	7	3,776	-224
12	O Projection GEMM	nvjet_sm100_tst_64x8	23	5,568	nvjet_sm100_tst_64x8	7	5,792	+224

Table A1: Attention phase matched operations. vS/sS = stream ID. Delta = SGL - vLLM.

Post-Attention AllReduce + Norm (Group 13)

Gr p	Logical Operation	vLLM Kernel	vS	vLLM D(ns)	SGL Kernel	sS	SGL D(ns)	Delta
13	Post-Attn AllReduce + RMSNorm + Residual	allreduce_fusion_kernel_oneshot_lamport	23	7,904	cross_device_reduce_1stage + fused_add_rmsnorm (COMBINED)	7	11,456	+3,552

MoE Phase - Routed Experts (Groups 14-21)

Gr p	Logical Operation	vLLM Kernel	vS	vLLM D(ns)	SGL Kernel	sS	SGL D(ns)	Delta
14	MoE Router Gate GEMM	router_gemm_kernel	23	2,880	router_gemm_kernel	3488	2,848	-32
15	MoE Workspace Zero Fill (vLLM ONLY)	vectorized_elementwise (FillFunctor)	23	2,368	---	--	---	N/A
16	MoE Input FP4 Quant	cvt_fp16_to_fp4	23	2,784	quantize_with_block_size	3488	2,816	+32
17	MoE Routing Data Copy (vLLM ONLY)	vectorized_elementwise (bfloat16_copy)	23	1,984	---	--	---	N/A
18	MoE TopK Routing	routingIndicesBlock	23	7,808	routingIndicesBlock	3488	7,264	-544
19	MoE Gate+Up GEMM + SwiGLU (FP4)	bmm_E2m1_t128x8x512	23	10,304	bmm_E2m1_t128x8x512	3488	9,504	-800
20	MoE Down GEMM (FP4)	bmm_Bfloat16_E2m1_t128x8x256	23	7,648	bmm_Bfloat16_E2m1_t128x8x256	3488	7,808	+160
21	MoE Finalize (Unpermute+Reduce)	finalizeKernel	23	4,064	finalizeKernel	3488	2,688	-1,376

Table A2: MoE routed expert operations. Note vLLM runs these on main stream (23); SGLang runs them on alt stream (3488).

MoE Phase - Shared Expert (Groups 22-28, parallel stream)

Gr p	Logical Operation	vLLM Kernel	vS	vLLM D(ns)	SGL Kernel	sS	SGL D(ns)	Delta
22	Shared Expert Buf Init (vLLM ONLY)	vectorized_elementwise (FillFunctor)	3649	1,408	---	--	---	N/A
23	Shared Gate+Up FP4 Quant	cvt_fp16_to_fp4	3649	1,824	cvt_fp16_to_fp4	7	2,272	+448
24	Shared Gate+Up GEMM (FP4 CUTLASS)	CUTLASS 128x64x128	3649	9,216	CUTLASS 128x128x256	7	11,392	+2,176
25	Shared SiLU+Mul	triton_poi_fused_mul_silu_slice	3649	2,496	act_and_mul_kernel	7	3,968	+1,472
26	Shared Down Buf Init (vLLM ONLY)	vectorized_elementwise (FillFunctor)	3649	1,344	---	--	---	N/A
27	Shared Down FP4 Quant	cvt_fp16_to_fp4	3649	1,472	cvt_fp16_to_fp4	7	1,920	+448
28	Shared Down GEMM (FP4 CUTLASS)	CUTLASS 128x128x256	3649	4,640	CUTLASS 128x64x128	7	8,831	+4,191

Table A3: Shared expert operations. vLLM TP-shards (smaller GEMMs, stream 3649); SGLang unsharded (larger GEMMs, main stream 7). Size differences explain large positive deltas for GEMM operations.

Output Combine + Final (Groups 29-30)

Gr p	Logical Operation	vLLM Kernel	vS	vLLM D(ns)	SGL Kernel	sS	SGL D(ns)	Delta
29	Add Shared + Routed Outputs (+ scale)	triton_poi_fused_add_mul	23	1,631	vectorized_elementwise (CUDAFunctor_add)	7	1,504	-127

Gr p	Logical Operation	vLLM Kernel	vS	vLLM D(ns)	SGL Kernel	sS	SGL D(ns)	Delta
30	MoE Output AllReduce (SGL ONLY)	---	--	---	cross_device_reduce_1stage	7	6,656	N/A

Table A4: Output combine. vLLM fuses the block-end AllReduce into the NEXT block's input RMSNorm (Group 1); SGLang runs it explicitly (Group 30).

Per-Stream Compute Totals

Phase	vLLM Stream 23	vLLM Stream 3649	SGL Stream 7	SGL Stream 3488
Attention phase	60,382 ns	0 ns	56,976 ns	1,568 ns
MoE phase	41,471 ns	22,400 ns	37,743 ns	32,928 ns
Total	101,853 ns	22,400 ns	94,719 ns	34,496 ns
Grand total	124,253 ns		129,215 ns	

Table A5: Per-stream compute breakdown. SGLang has 4.0% more total compute but 48% more overlap, yielding 7.9% lower wall time.

Key Fusion Differences

#	Fusion Pattern	vLLM	SGLang
F1	AllReduce + RMSNorm + Residual	FUSED (Lamport kernel)	SEPARATE (2 kernels)
F2	Q-A RMSNorm	2 Triton kernels (torch.compile)	1 FlashInfer kernel
F3	KV-A Norm + RoPE	FUSED Triton kernel	SEPARATE norm + fused RoPE+FP8
F4	Q concat + softmax scale	SEPARATE Triton kernel	FUSED inside MLA attention
F5	Shared expert SiLU+Mul	torch.compile Triton	sgl-kernel CUDA kernel
F6	Output add + scaling	FUSED (add+mul Triton)	SEPARATE add (scale in MoE)

Table A6: Kernel fusion strategy comparison.

Appendix B: Source References

vLLM Source Files (Target)

Source File	Relevance
vllm/model_executor/models/deepseek_v2.py	DeepseekV2MLP, shared expert construction, MoE layer
vllm/model_executor/layers/fused_moe/experts/trtllm_nvfp4_moe.py	TRTLLM NVFP4 MoE integration, workspace management, routing bias
vllm/model_executor/layers/fused_moe/runner/moe_runner.py	MoE runner orchestration, stream management
vllm/model_executor/layers/fused_moe/runner/shared_experts.py	Shared expert auxiliary stream execution
vllm/model_executor/layers/attention/mla_attention.py	MLA attention, Q concat, W_UK layout, decode path
vllm/v1/attention/backends/mla/flashinfer_mla.py	FlashInfer MLA backend, forward_mqa
vllm/compilation/passes/fusion/allreduce_rms_fusion.py	AllReduce+RMSNorm fusion compilation pass
vllm/config/vllm.py	AllReduce fusion enable conditions

SGLang Source Files (Baseline)

Source File	Relevance
python/sglang/srt/models/deepseek_v2.py	Shared expert tp_size=1, forward_normal_dual_stream
python/sglang/srt/models/deepseek_common/.../forward_mla.py	MLA forward, separate Q component passing
python/sglang/srt/layers/moe/moe_runner/flashinfer_trtllm.py	FlashInfer TRTLLM MoE wrapper, torch.empty output

External References

Reference	Description
FlashInfer Issue #2909	Routing bias dtype requirement (bfloat16)
vLLM commit fe9c3d6c5f66c	Target framework version (v0.19.2rc1)
SGLang commit 612785ffdcaf3	Baseline framework version

Implementation Priority & Sequencing

Phase	Improvement	Effort	Expected Saving	Cumulative
Phase 1	IMP-2a: Pre-convert routing bias	1 day	-1,984 ns	-1,984 ns
Phase 1	IMP-1: Unsharded shared expert + swap	3-5 days	-7,232 ns	-9,216 ns
Phase 2	IMP-2b: Eliminate workspace zero-fill	2-3 days	-2,368 ns	-11,584 ns
Phase 2	IMP-3: Optimize Q concat+scale	1-2 days	-1,696 ns	-13,280 ns
Phase 3	IMP-4: Attention micro-opts	2-3 days	-350 ns	-13,630 ns

Table B3: Implementation phases. Total estimated effort: 9-14 engineering days. After all improvements + POS-1 advantage, vLLM estimated at ~84,289 ns vs SGLang 90,144 ns (~6.5% faster).